

[BUG] sum_expand does not expand properly

<https://github.com/PennyLaneAI/pennylane/issues/3891>

ROW 27

[BUG] sum_expand does not expand properly #3891

New issue



Closed



timmysilv opened on Mar 13, 2023

...

Expected behavior

I expect the post-processed results of `sum_expand` to be equivalent to the inputted circuit.

Actual behavior

Some operands are dropped that should not be. See the source code section for a minimal example of the issue. Note that an equivalent `Hamiltonian` does not drop the `Z0Z1` term when passed to `qml.transforms.hamiltonian_expand`.

Additional information

No response

Source code

```
import pennylane as qml

op = qml.sum(qml.prod(qml.PauliX(0), qml.PauliZ(1)), qml.prod(qml.PauliZ(0), qml.PauliZ(1)))
tape = qml.tape.QuantumScript(measurements=[qml.expval(op)])
tapes, _ = qml.transforms.sum_expand(tape)
print([t.measurements for t in tapes])
# [[expval(PauliX(wires=[0]) @ PauliZ(wires=[1]))]] # the Z0Z1 term disappeared!
```



Tracebacks

No response

System information

```
Name: PennyLane
Version: 0.30.0.dev0
Summary: PennyLane is a Python quantum machine learning library by Xanadu Inc.
Home-page: https://github.com/XanaduAI/pennylane
Author:
Author-email:
License: Apache License 2.0
Location: /Users/matthews/.pyenv/versions/3.9.13/envs/pl/lib/python3.9/site-packages
Editable project location: /Users/matthews/src/github.com/PennyLaneAI/pennylane
Requires: appdirs, autograd, autoray, cachetools, networkx, numpy, pennylane-lightning, requests, retworkx, scikit-learn
Required-by: PennyLane-Lightning

Platform info:      macOS-13.2.1-arm64-arm-64bit
Python version:     3.9.13
Numpy version:      1.23.5
Scipy version:      1.10.0
Installed devices:
- default.gaussian (PennyLane-0.30.0.dev0)
- default.mixed (PennyLane-0.30.0.dev0)
- default.qubit (PennyLane-0.30.0.dev0)
- default.qubit.autograd (PennyLane-0.30.0.dev0)
- default.qubit.jax (PennyLane-0.30.0.dev0)
- default.qubit.tf (PennyLane-0.30.0.dev0)
- default.qubit.torch (PennyLane-0.30.0.dev0)
- default.qutrit (PennyLane-0.30.0.dev0)
- null.qubit (PennyLane-0.30.0.dev0)
- lightning.qubit (PennyLane-Lightning-0.29.0.dev11)
```



Existing GitHub issues

☒ I have searched existing GitHub issues to make sure the issue does not already exist.

Assignees

No one assigned

Labels

bug 🐛

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

Customize



Subscribe

You're not receiving notifications from this thread.

Participants



Existing GitHub issues

☒ I have searched existing GitHub issues to make sure the issue does not already exist.



timmysilv added **bug** on Mar 13, 2023



albi3ro on Mar 13, 2023

Contributor ...

This is due to a problem with `MeasurementProcess.hash`

```
>>> t1 = qml.prod(qml.PauliX(0), qml.PauliZ(1))
>>> t2 = qml.prod(qml.PauliZ(0), qml.PauliZ(1))
>>> qml.expval(t1).hash == qml.expval(t2).hash
True
```



👍 1

albi3ro mentioned this on Mar 14, 2023

[Fix MeasurementProcess.hash and CompositeOp.hash #3898](#)



timmysilv on Apr 11, 2023

Contributor Author ...

this has been fixed



timmysilv closed this as **completed** on Apr 11, 2023

Fix MeasurementProcess.hash and CompositeOp.hash #3898

<> Code

Merged albi3ro merged 13 commits into master from measurement-process-hash on Mar 20, 2023

Conversation 37 Commits 13 Checks 0 Files changed 27 +406 -231

 albi3ro commented on Mar 14, 2023 • edited Contributor

This PR has expanded to fixing a lot of minor details. It can be broken up into multiple PR's if so requested by reviewers.

Solves #3891

[sc-34965]

sum_expand was treating measurement processes as equal when they were not actually equal. The problem was due to MeasurementProcess.hash. Now if the measurement has an observable, it uses the observables hash.

This change uncovered problems with Sum.hash and Prod.hash. The hash was first sorting the operations, so that would be the same even if the terms were in a different order, as long as the terms commuted. Sorting was failing when the wires had mixed types, like 0 and "a".

Since the terms in a Sum are always commutative, we use a frozen set instead of sorting.

Prod now compares the string version of the wire labels, as strings can always be compared with <. This will fail if the operator has both 0 and "0" as wire labels, but if someone does that they are just asking for trouble.

Since this involved trying out possibilities for MeasurementProcess, I uncovered other things that needed fixing or tidying. The smaller changes in this PR include:

- Adding type hints
- Docstrings now say Operator instead of Observable.
- Only allowing the MeasurementProcess to be initialized with the inputs it actually accepts. For example, you can no longer initialize a MutualInfoMP with an observable or eigenvalues.
- Child measurements needed improved hash, repr, or copies for special properties like log_base.
- Purity was added as an ObservableReturnType. Before, PurityMP.return_type.value was raising an error, so its repr was failing.

Reviewers



Gottmann



rmoyard

✓



timmysilv

✓

Assignees

No one assigned

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.